

Attention Is All You Need

1. 서론 및 제안 배경 (Introduction)

기존 모델의 한계 (RNN/CNN)

- 기존의 시퀀스 모델링은 주로 **RNN(Recurrent Neural Network)**, **LSTM**, **GRU**에 기반함.
- **순차적 연산의 문제**: RNN은 이전 hidden state를 받아 현재 상태를 계산해야 하므로, 시간 순서대로 계산이 이루어져야 함. 이는 학습 시 병렬화를 불가능하게 만들.
- 긴 시퀀스 길이에서 메모리 제약이 발생하며, 문장 간의 거리가 멀어질수록 학습이 어려움.

Transformer의 제안

- **재귀(Recurrence)**와 **합성곱(Convolution)**을 완전히 배제하고, 오직 **어텐션** 메커니즘에만 의존하는 새로운 아키텍처인 **Transformer**를 제안함.
- **장점**:
 - 입출력 간의 전역적 의존성을 상수 시간 안에 파악 가능.
 - **병렬화 극대화**: 훈련 속도가 획기적으로 빠름.
 - 번역 품질에서 기존 SOTA를 능가함.

2. 관련 연구 배경 (Background)

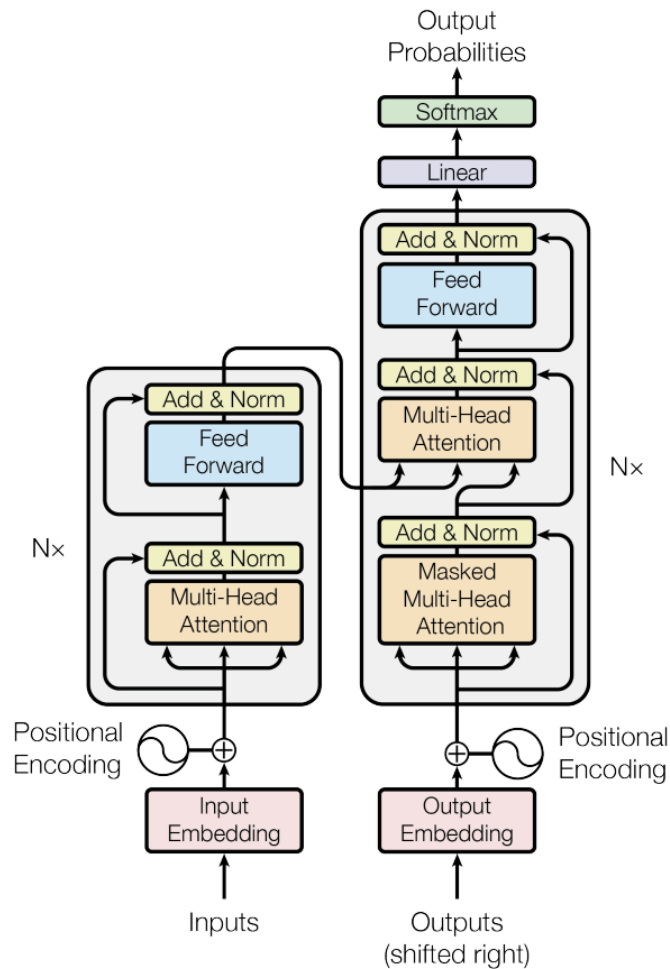
기존 CNN 모델과의 차이점

- ByteNet, ConvS2S 등은 합성곱 신경망(CNN)을 사용하여 병렬 처리를 시도했으나, 멀리 떨어진 위치의 정보를 연결하려면 층(Layer)을 깊게 쌓아야 함.
- **Transformer**: 어텐션을 사용하여 위치 간 거리에 상관없이 **상수(Constant) 횟수의 연산**으로 모든 위치를 연결함.

Self-Attention

- 단일 시퀀스 내의 서로 다른 위치들을 연관 지어 시퀀스의 표현을 계산하는 방식.
- 기존에도 독해 등에서 사용되었으나, Transformer는 **RNN이나 Convolution 없이 오직 Self-Attention만으로 입출력을 변환하는 최초의 모델**.

3. 모델 아키텍처 (Model Architecture)



Transformer는 경쟁력 있는 신경망 모델들이 따르는 **인코더-디코더(Encoder-Decoder)** 구조를 따름.

- **Encoder:** 입력 시퀀스 $(x_1, \dots, x_n)$ 를 연속적인 표현 $z = (z_1, \dots, z_n)$ 로 매핑.
- **Decoder:** z 를 받아 출력 시퀀스 $(y_1, \dots, y_m)$ 를 한 번에 하나씩 생성.
- **Auto-regressive:** 이전에 생성된 심볼들을 다음 생성을 위한 추가 입력으로 사용함.

3.1 Encoder and Decoder Stacks

Transformer는 인코더와 디코더 모두에 **Stacked Self-Attention**과 **Point-wise Fully Connected Layers**를 사용함.

Encoder (인코더)

- 구조: $N=6$ 개의 동일한 레이어(Layer)를 쌓아 올린 형태.
- 각 레이어의 구성 (2개의 Sub-layers):

1. Multi-Head Self-Attention

2. Position-wise Fully Connected Feed-Forward Network

- 연결 방식:
 - 각 서브 레이어 주위에 잔차 연결을 적용하고, 그 후 층 정규화를 수행함.
- 차원: 잔차 연결을 용이하게 하기 위해, 모든 서브 레이어와 임베딩 층의 출력 차원을 통일함.

Decoder (디코더)

- 구조: 인코더와 마찬가지로 $N=6$ 개의 동일한 레이어를 쌓음.
- 각 레이어의 구성 (3개의 Sub-layers):

1. Masked Multi-Head Self-Attention: 디코더의 자기 어텐션.

2. Multi-Head Attention: 인코더의 출력을 활용하는 어텐션.

3. Position-wise Fully Connected Feed-Forward Network

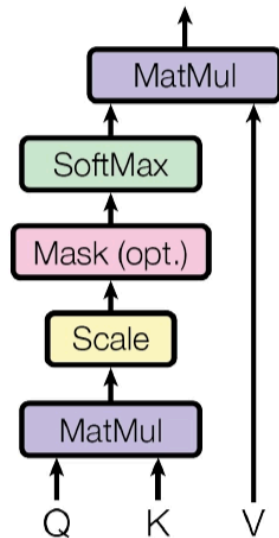
- Masking의 중요성:
 - 디코더의 Self-Attention은 현재 위치보다 **이후에 있는 위치를 참조하지 못하도록** 수정됨.
 - 이는 예측 시 미래의 정보를 미리 보는 것을 방지하여 i 번째 위치의 예측이 오직 i 보다 작은 위치의 출력 들에만 의존하게 함.
- 인코더와 동일하게 잔차 연결 및 Layer Normalization 적용

3.2 Attention

Attention 함수는 Query(Q)와 Key(K)-Value(V) 쌍을 Output으로 매핑하는 과정임.
Output은 Value들의 가중합으로 계산됨.

3.2.1 Scaled Dot-Product Attention

Scaled Dot-Product Attention



- 구조 및 수식

- 입력은 차원 d_k 의 Query와 Key, 차원 d_v 의 Value로 구성됨.
- Query와 모든 Key의 내적을 구하고, 이를 $\sqrt{d_k}$ 로 나눈 뒤 Softmax를 적용하여 Value에 대한 가중치를 구함.
- 수식: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$.

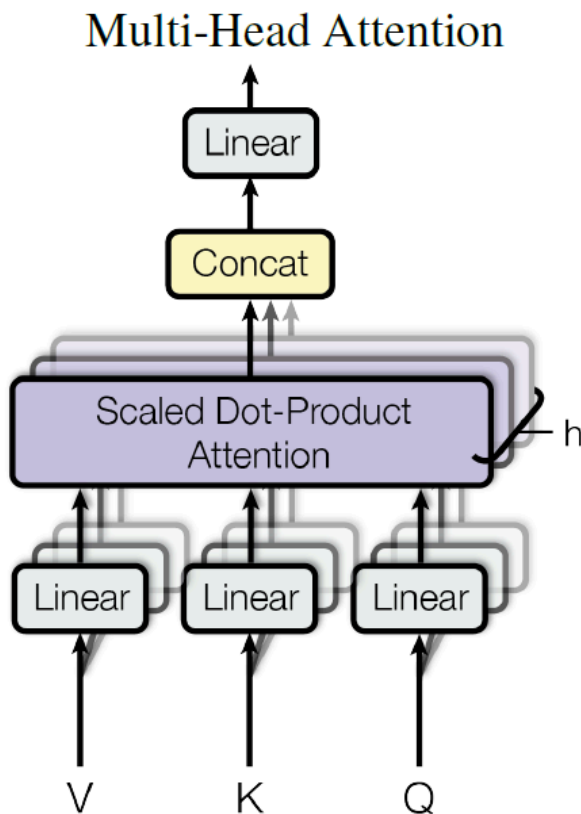
- Additive vs Dot-Product Attention

- 이론적인 복잡도는 비슷하지만, Dot-Product 방식이 최적화된 행렬 곱 연산 코드를 사용할 수 있어 속도가 더 빠르고 메모리 효율적임.

- Scaling ($\frac{1}{\sqrt{d_k}}$)을 하는 이유

- d_k 값이 커질수록 Dot-product(QK^T)의 magnitude 매우 커짐.
- 값이 커지면 Softmax 함수의 Gradient가 extremely small gradients로 빠지게 됨.
- 이를 방지하기 위해 $\sqrt{d_k}$ 로 나누어 스케일링을 수행함.

3.2.2 Multi-Head Attention



- 개요
 - d_{model} 차원의 Key, Value, Query로 단일 Attention을 수행하는 것보다, 서로 다른 h 개의 학습된 선형 투영을 통해 차원을 줄여 병렬로 처리하는 것이 유리함.
- 동작 과정

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

1. Query, Key, Value를 각각 d_k, d_k, d_v 차원으로 h 번 선형 투영함.
2. 투영된 각 버전에서 병렬로 Attention을 수행 (Output 값 도출).

3. 결과값들을 이어 Concat한 후, 다시 한번 선형 투영하여 최종 결과 생성.

- 효과
 - 모델이 서로 다른 위치에 있는 서로 다른 표현 부분 공간의 정보에 동시에 집중할 수 있게 함. (단일 Head에서는 평균화로 인해 이 정보가 희석됨)
- 설정
 - 논문에서는 $h = 8$ 을 사용. 각 Head의 차원은 $d_k = d_v = d_{model}/h = 64$.
 - 차원을 줄였기 때문에 총 Cost는 Single-head Attention과 비슷함.

3.2.3 Applications of Attention in our Model

Transformer는 Multi-Head Attention을 세 가지 다른 방식으로 활용함.

1. Encoder-Decoder Attention Layers

- Query: 이전 디코더 레이어에서 옴.
- Key, Value: 인코더의 최종 Output에서 옴.
- 역할: 디코더의 모든 위치에서 입력 시퀀스의 모든 위치를 참조할 수 있게 함. (Seq2Seq의 전형적인 Attention 방식 모방)

2. Encoder Self-Attention Layers

- Query, Key, Value: 모두 인코더의 이전 레이어 출력에서 옴.
- 역할: 인코더의 각 위치가 이전 레이어의 모든 위치를 참조하여 문맥을 파악함.

3. Decoder Self-Attention Layers

- Query, Key, Value: 모두 디코더의 이전 레이어 출력에서 옴.
- Masking: 디코더는 자기 회귀 속성을 유지해야 함.
- i 번째 단어를 예측할 때 $i + 1$ 번째 이후의 단어, 즉 미래의 정보를 참조하면 안 됨.
- 구현: Scaled Dot-Product Attention 내부에서 Softmax 입력 값을 $-\infty$ 로 마스킹 처리하여, 미래 위치와의 연결을 끊음.

3.3 Position-wise Feed-Forward Networks

단어별로 독립적으로 각 단어의 정보를 정제하는 신경망. self-attention에서 문맥 정보를 반영했다면, FFN에서는 각 단어 벡터를 정교하게 처리한다.

- 구조 : 입력 벡터 $\rightarrow$ Linear (W_1, b_1) $\rightarrow$ ReLU $\rightarrow$ Linear (W_2, b_2) $\rightarrow$ 출력 벡터
 - 즉, $FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$
 - ReLU = $\max(0, x)$
 - 먼저 선형 변환을 통해 x 를 더 넓은 차원으로 확장했다가 ReLU로 비선형성을 추가하고, 다시 선형변환하여 차원을 축소한다. (ex) 512 $\rightarrow$ 2048 $\rightarrow$ 512차원) 이때, 입력과 출력의 차원(d_{model})은 512로 같으며, 중간에 가중치행렬에 의해 $d_{ff} = 2048$ 차원으로 확장되는 것이다.
- encoder와 decoder 모두에 존재한다. encoder, decoder 각각에 있는 N개의 layer는 각각 다른 parameter를 사용하며, 같은 layer에서는 모든 단어 위치에 대해 같은 parameter를 사용한다.
- FFN은 종속성 없이 단어마다 독립적으로 작동하므로 "two convolutions with kernel size 1"라고도 불린다. 즉 단어벡터 하나씩만 보고 처리하는 과정을 두 번(선형변환)한다는 것이다.

3.4 Embeddings and Softmax

- 다른 시퀀스변환 모델 (..) 과 마찬가지로 임베딩 사용 : 입출력 토큰을 d_{model} 크기의 벡터로 변환 ex) (I) $\rightarrow$ [0.2, 0.4, ..., 0.7] 512차원
- decoder 마지막 단계에서 선형변환 $\rightarrow$ softmax 함수로 출력벡터를 예측되는 다음 토큰의 확률로 변환
 - linear 변환 : softmax에 넣을 logit을 만드는 변환. decoder에서 생성된 벡터를 훨씬 더 큰 벡터로 투영한다. ex) 모델이 학습 데이터셋에서 학습한 10000개의 고유한 영어 단어를 알고 있으면 10000개 셀로 구성된 Logit vector을 만들며, 각 셀은 단어의 score를 담고 있다.
 - softmax : 모든 단어들에 대한 score를 확률로 변환한다. 가장 높은 확률값이 다음단어!

- 두 embedding layer (encoder에 처음 단어를 정수에서 고차원 벡터화시키는 Layer와, 출력 전에 decoder의 출력벡터를 단어로 변환하는 선형 변환 layer를 뜻함) 는 같은 가중치행렬을 쓴다.
 - embedding layer들에서는 이 가중치행렬에 $\sqrt{d_{model}}$ 을 곱한다.
 - embedding이 너무 작은 값이 되면 학습이 안될 수 있기 때문에 스케일링하는 작업!

3.5 Positional Encoding

🔥 RNN, LSTM과 달리 입력되는 문장을 순차적으로 처리 하지 않고, 입력된 문장을 병렬로 한번에 처리함. 연산은 빠르지만 단어의 위치(순서)를 알수 없는 문제 → 위치를 나타내는 인코딩을 추가하자!

- positional encoding의 차원 : 입력차원과 같음 (d_{model})
- 모든 위치값은 시퀀스 길이, input에 상관없이 동일한 식별자를 가져야한다. 즉 시퀀스 변경되어도 positional encoding은 동일하다.
- 논문에서 제시한 positional encoding
이렇듯 첫번째 차원부터 마지막 차원의 벡터값을 서로 다른 frequency를 가진 sin & cos 을 번갈아 계산하면서 채우면 충분히 다른 encoding 값을 지닐 수 있게 된다.

$$\text{짝수 차원 } PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$\text{홀수 차원 } PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

- pos : position (몇번째 단어인지) / i: 차원 (단어 임베딩에서 몇번째 차원인지)
- 이때 주기함수의 파장(wavelength)은 2π 부터 $10000 \cdot 2\pi$ 까지 기하급수적 증가. 즉 뒷쪽 차원으로 갈수록 (i가 커질수록) 파장이 길어진다. 이는 곧 뒷쪽 차원으로 갈수록 위치가 많이 달라져도 출력값이 비슷하게 유지되어, 전체 구조에서 거시적인 위치 패턴을 인식할 수 있다는 뜻이다.
- ⚠ 꼭 sin,cos 기반 고정 인코딩이 아니더라도 위치정보를 학습하는 positional embedding도 잘 작동한다.
- 그러나 논문에서 sin,cos 기반 인코딩 선택한 이유는 두가지다.
 - 첫째, 위치 차이 k로 고정되어있을때 pos+k 위치의 positional encoding을 pos 위치의 positional encoding으로 계산 가능하다. 즉 상대적 위치 정보 학습이 쉽다.

(ex) 단어 A와 B 사이 3칸 차이가 있다고 하면, A positional encoding을 알면 B도 선형적으로 계산 가능)

- 둘째로, 학습 중 보지 못한, 길이가 더 긴 문장에서도 일반화될 수 있기 때문이다.

4 Why Self-Attention

왜 RNN, CNN 계열이 아닌 Self-Attention을 사용해야 하는가?

1. 각 레이어마다 계산복잡도가 줄어든다. 즉, 연산을 많이 하지 않는다.
2. 단어 순서 따라 계산해야하는 recurrence를 없애서 병렬적 처리가 가능하다.
3. long-range dependency를 잘 처리할 수 있다. 시퀀스 변환 task에서는 신호가 앞뒤로 이동할때 통과하는 경로가 짧아야, 앞에 있는 단어와 뒤에 있는 단어 사이의 관계(장거리 의존성)를 모델이 잘 기억할 수 있다.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

n: 시퀀스 길이(단어수) / d : d_{model} (임베딩 차원) / k : CNN 커널 사이즈 / r : self-attention이 참조하는 주변 토큰 수

- 계산복잡도 : $n < d$ 일 때 Self-Attention이 RNN보다 빠르다. 대부분의 경우 시퀀스길이보다 임베딩 차원이 크므로, self-attention이 더 빠르다는 뜻.
- 순차연산 : Self-Attention 은 한번만에 연산할 수 있지만 (병렬처리), RNN은 시퀀스 길이만큼 연산이 필요하다.
- Self-Attention (restricted)는 시퀀스가 너무길때 전체에 attention을 적용하지 않고 출력위치 주변의 r개 단어만 보는 layer이다. 이럴 경우 self-attention의 최대 경로 길이가 $O(1)$ 에서 $O(n/r)$ 로 늘어난다.

ex) 1번 단어가 2~6번까지밖에 못 보는데 10번 단어와 정보 교환하려면 → 여러 단계 필요

- CNN에서 $k < n$ 이면 하나의 CNN layer로 모든 입-출력 위치를 연결 불가능

ex) 1번 단어는 1~3번 단어까지만 보게되면 , 1번 단어의 정보가 8번 단어까지 바로 닿지 않아 전범위 정보 전파 X.

→ 모든 층을 연결하려면 여러 층이 필요함! 일반적으로는 $O(n/k)$ 이나 maximum path length를 늘리려면 확장된 CNN은 $O(\log_k n)$ 층이 필요하다.

- 또한 계산복잡도 측면에서, CNN이 일반적으로 RNN보다 계산비용이 k만큼 크다 (필터를 여러 위치에 적용하므로)
- Seperable Convolution은 계산복잡도를 기존 CNN의 $O(knd^2)$ 에서 $O(knd + nd^2)$ 로 크게 줄일 수 있다. 그러나 $k = n$ 이 되더라도 결국 복잡도는 Self-Attention + FeedForward Layer와 비슷해져서, 굳이 CNN 쓸 이유 X
 - Seperable Convolution : 채널 별로 필터 적용하고, 나중에 결과를 합치는 방식.

이외 특징

- 어떤 단어를 주의하고 있는지 해석 가능성이 더 높다. (ex) heatmap 시각화)
- 개별 attention head들은 서로 다른 역할을 함
 - ex) head 1은 동사-주어 관계를 집중해서 보고 head2는 명사구 전체에 집중

5.1 Training Data and Batching

- 데이터셋: **WMT 2014 English-German**(약 450만 문장쌍)으로 학습.
- 토큰나이징: **BPE(Byte-Pair Encoding)** 사용
 - En-De: 공유(source-target) vocab 약 37,000.
- En-Fr 실험:
 - 더 큰 **WMT 2014 English-French**(약 3,600만 문장) 사용.
 - vocab 약 32,000 word-piece로 분할.
- 배치 구성:
 - 비슷한 시퀀스 길이끼리 묶어 배치.
 - 각 학습 배치 ≈ 소스 토큰 25,000 + 타겟 토큰 25,000(총량 기준).

5.2 Hardware and Schedule

- 하드웨어: 한 대의 머신에서 **NVIDIA P100 GPU 8개**로 학습.
- Base 모델:
 - step time 약 **0.4초**
 - 총 **100,000 step** 학습 → 약 **12시간**
- Big 모델:
 - step time 약 **1.0초**
 - 총 **300,000 step** 학습 → 약 **3.5일**

5.3 Optimizer (학습률 스케줄)

- 옵티마이저: **Adam**
 - $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\varepsilon = 1e-9$
- Learning rate 스케줄("Transformer lr schedule"):
 - 초기 **warmup** 동안 선형 증가, 이후에는 **step**의 제곱근에 반비례($\approx 1/\sqrt{\text{step}}$)로 감소
 - warmup_steps = **4000**

5.4 Regularization

학습 중 정규화는 크게 다음을 사용:

Residual Dropout

- 각 sub-layer 출력에 dropout 적용 후 residual로 더하기 전에 사용.
- encoder/decoder에서 ****임베딩 합(토큰 임베딩 + positional encoding)****에도 dropout 적용.
- base 모델 dropout 비율: **Pdrop = 0.1**

Label Smoothing

- label smoothing 값: $\epsilon_{ls} = 0.1$
- 효과:
 - perplexity는 나빠질 수 있지만(모델이 덜 확신하게 됨),
 - 정확도 및 BLEU는 개선되는 경향.

6.1 Machine Translation

WMT14 En → De

- Transformer (big) 이 기존 최고 성능(앙상블 포함) 대비 **BLEU +2.0 이상** 향상.
- SOTA BLEU 28.4 달성.
- 학습: P100 GPU 8장에서 약 3.5일.
- base 모델도 기존 발표 모델/앙상블을 훨씬 적은 학습 비용으로 능가한다고 서술.

WMT14 En → Fr

- Transformer (big) BLEU 41.0.
- 기존 단일 모델들보다 성능이 높고, 이전 SOTA 대비 학습비용이 1/4 미만이라고 주장.
- En → Fr big 모델은 dropout을 Pdrop=0.1 사용(이전 값 0.3 대신).

추론/학습 세부(공통)

- 체크포인트 평균:
 - base: 마지막 5개 체크포인트 평균
 - big: 마지막 20개 체크포인트 평균
- Beam search:
 - beam size 4, length penalty $\alpha=0.6$
 - 최대 출력 길이: 입력 길이 + 50
- 학습 비용 비교:

- Table 2에서 문헌의 다른 구조들과 **번역 품질 + 학습비용**을 함께 비교.
- 학습 연산량은 **학습시간 × GPU 수 × GPU의 지속 단정밀 성능 추정치**로 FLOPs를 추정한다고 설명.

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

6.2 Model Variations (구성요소 영향 분석)

- Table 3(A): **head 수와 key/value 차원을 바꾸며(계산량은 유지) 성능 비교**
 - **single-head attention은 최적 설정 대비 BLEU 약 0.9 낮음**
 - **head가 너무 많아도 품질이 떨어지는 경향 언급**
- Table 3(B): attention의 key 크기 **d_k 를 줄이면 품질 하락**
 - 단순 dot-product보다 더 정교한 **compatibility 함수**가 도움될 가능성 시사
- (C)(D): **모델이 클수록 성능이 좋음**
- (D): **dropout이 과적합 방지에 유효**
- (E): sinusoidal positional encoding을 학습 **positional embedding**으로 바뀌어도 **base와 거의 동일한 결과**

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)				16						5.16	25.1	58
				32						5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)				positional embedding instead of sinusoids						4.92	25.7	
big	6	1024	4096	16			0.3		300K	4.33	26.4	213

6.3 English Constituency Parsing (영어 구문 분석)

- 목표: Transformer가 번역 외 과제에도 **일반화**되는지 평가.
- 과제 특성:
 - 출력이 **강한 구조 제약**을 받고 입력보다 **구조적으로 더 길고 복잡**.
 - 기존 RNN seq2seq는 **소규모 데이터에서 SOTA 달성**이 어려웠다는 배경 설명.

실험 설정

- 모델: **4-layer Transformer, $d_{\text{model}} = 1024$**
- 데이터:
 - Penn Treebank의 **WSJ 구간** (학습 문장 약 **40K**)
 - 반지도(semi-supervised): BerkeleyParser로 만든 **고신뢰 코퍼스 ~17M 문장** 추가

- vocab:
 - WSJ-only: **16K tokens**
 - semi-supervised: **32K tokens**
- 하이퍼파라미터 튜닝은 제한적으로(dropout/learning rate/beam 정도만), 나머지는 En → De base 설정 유지.
- 추론:
 - 최대 출력 길이 **입력 + 300**
 - beam size **21**, length penalty $\alpha=0.3$

결과 요지

- 과제 특화 튜닝이 거의 없는데도 **매우 잘 동작**.
- 기존 보고 모델들보다 **대체로 더 좋은 결과**(예외: RNNG).
- 특히 **WSJ 40K만 학습해도 Transformer가 BerkeleyParser보다 우수**하다고 서술.

Table 4: The Transformer generalizes well to English constituency parsing (Results are on Section 23 of WSJ)

Parser	Training	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	WSJ only, discriminative	88.3
Petrov et al. (2006) [29]	WSJ only, discriminative	90.4
Zhu et al. (2013) [40]	WSJ only, discriminative	90.4
Dyer et al. (2016) [8]	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013) [40]	semi-supervised	91.3
Huang & Harper (2009) [14]	semi-supervised	91.3
McClosky et al. (2006) [26]	semi-supervised	92.1
Vinyals & Kaiser et al. (2014) [37]	semi-supervised	92.1
Transformer (4 layers)	semi-supervised	92.7
Luong et al. (2015) [23]	multi-task	93.0
Dyer et al. (2016) [8]	generative	93.3